Assignment -05

Name: Lamisa Aziz

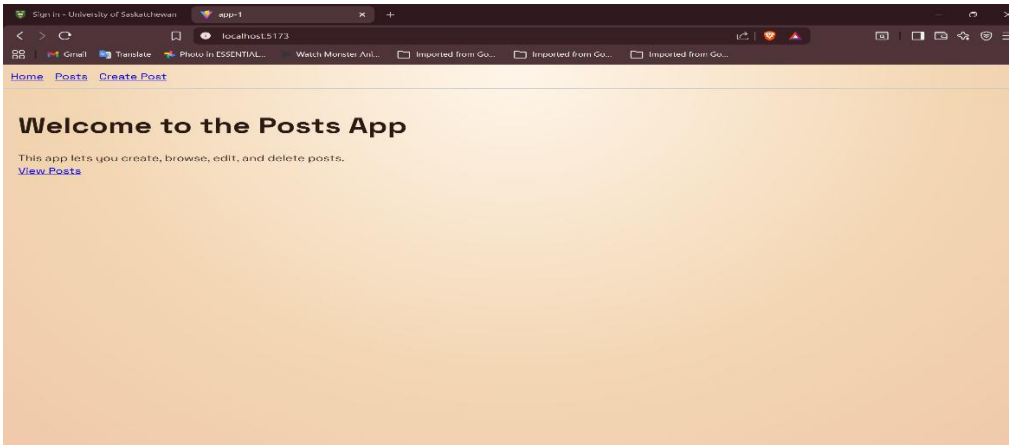
NSID: zlq765

Student ID: 11380653

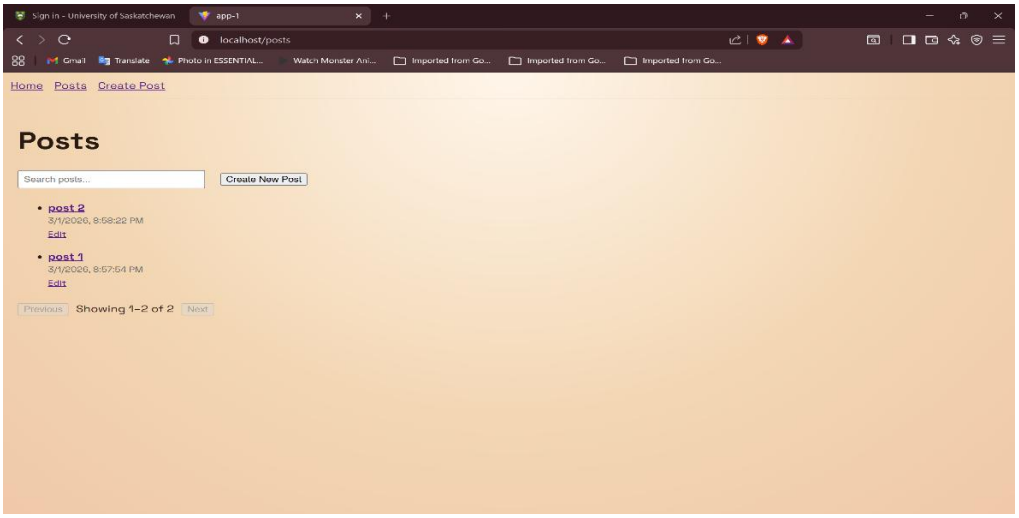
6. *Part C: Test Report*

6.1 Screenshots

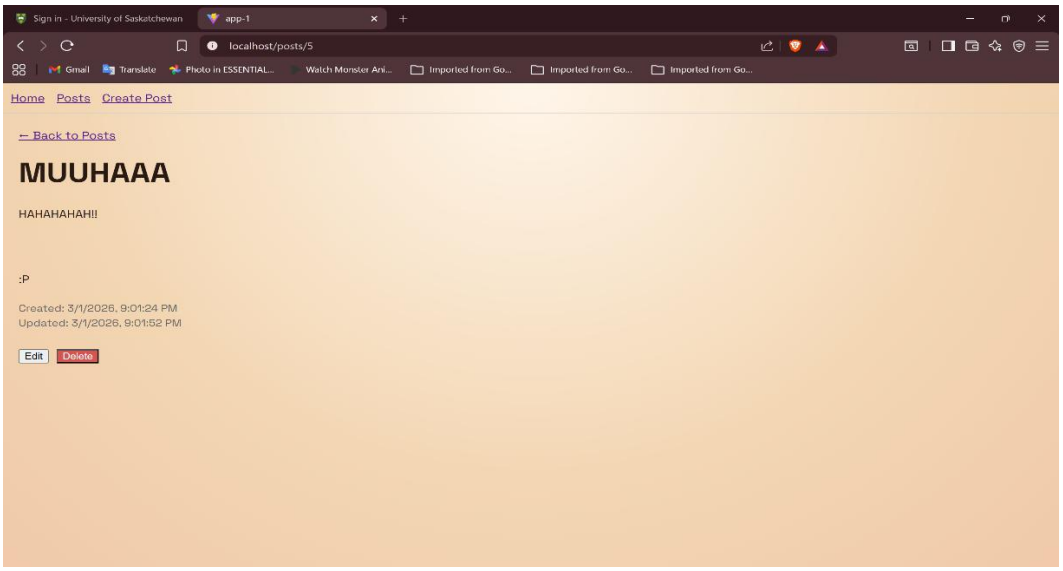
• Landing page



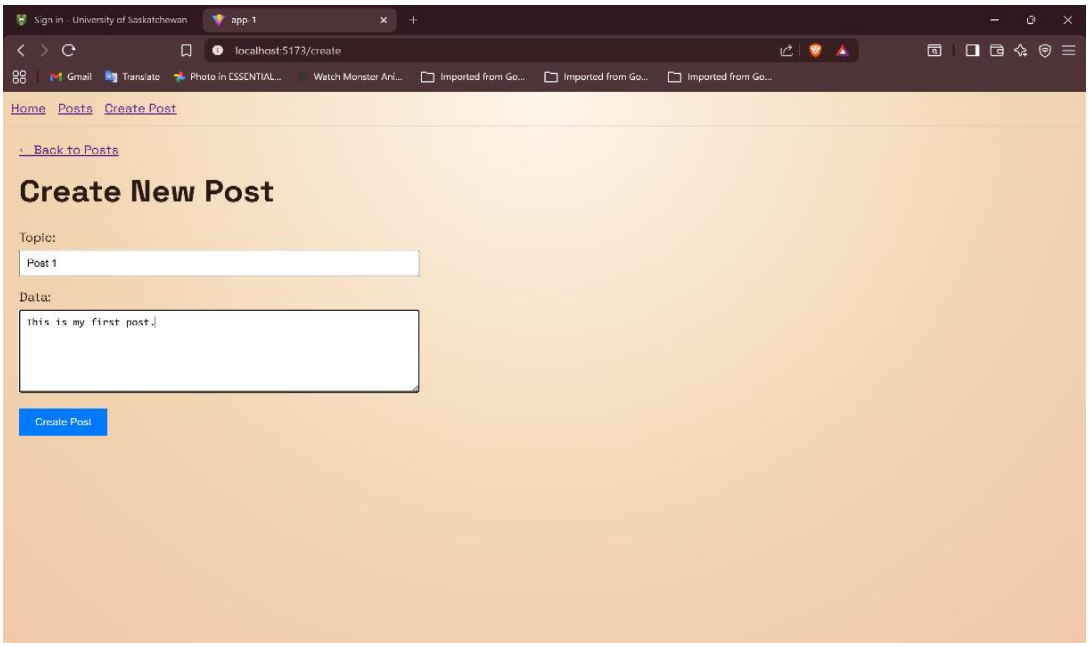
• Posts list page (showing pagination/search controls)



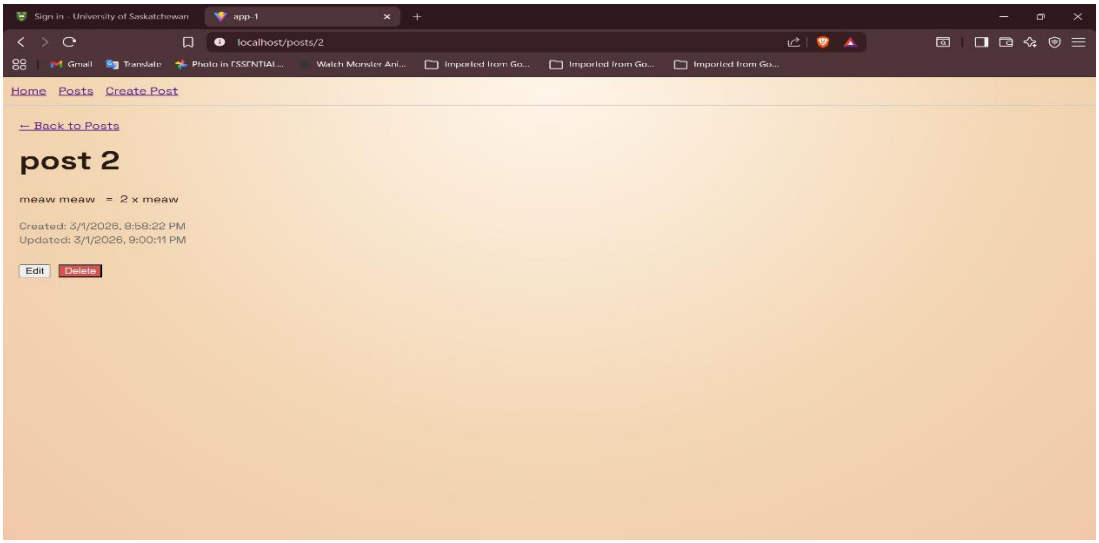
- Post details page

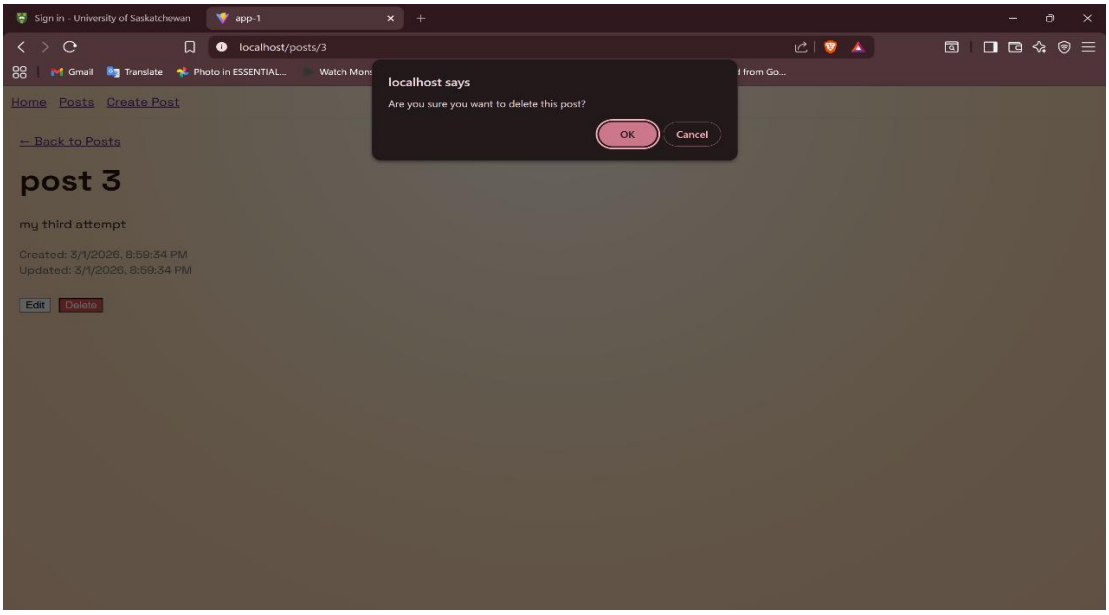


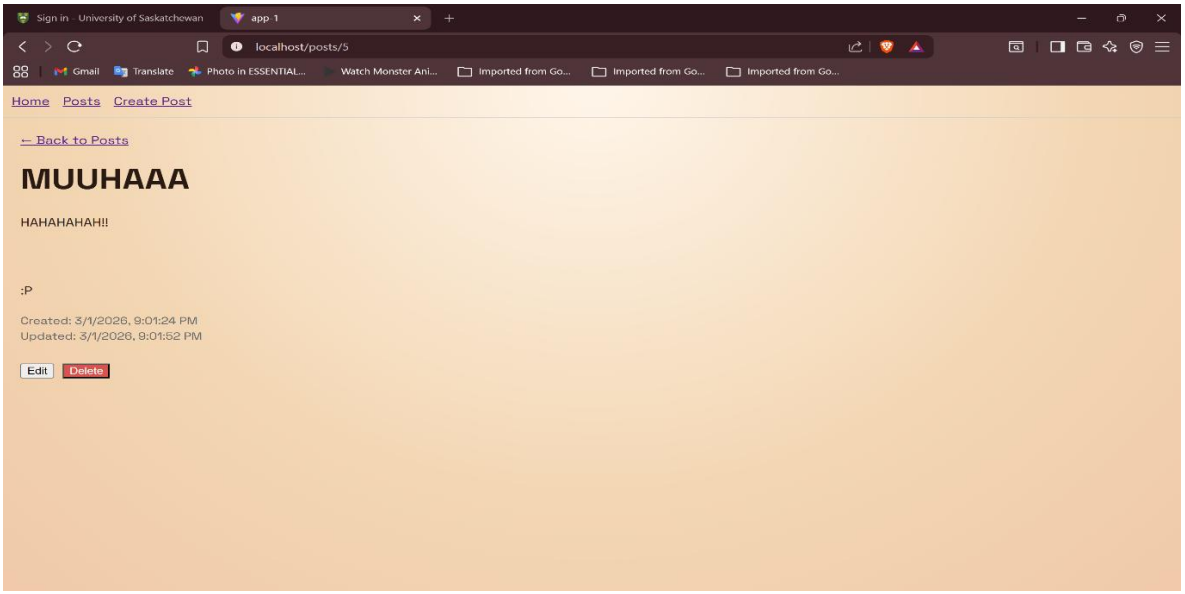
- Create post page



- Edit post page







6.2 Overall Testing Approach

• Backend testing: endpoints, status codes, JSON format, validation, not-found cases:

Backend Testing

For backend testing, I manually tested all REST endpoints using the browser and the Network tab in Developer Tools. I tested:

- GET /api/posts (list with pagination)
- GET /api/posts/:id
- POST /api/posts
- PUT /api/posts/:id
- DELETE /api/posts/:id

I verified that successful requests returned the correct status codes (200 OK for GET/PUT/DELETE and 201 Created for POST). I also tested validation by trying to submit empty topic or data fields and confirmed that the server returned 400 Bad Request.

To test not-found cases, I accessed a non-existing post ID (for example /api/posts/9999) and confirmed the server returned 404 Not Found.

I also checked that all responses were valid JSON. During debugging, I encountered an issue where HTML was returned instead of JSON, which helped me identify a routing/proxy problem between frontend and backend.

• Frontend testing: navigation, fetch calls, create/edit/delete flow, error handling:

Frontend Testing

For frontend testing, I tested navigation between all pages (posts list, create post, post details, and edit post) to ensure routing worked correctly.

I verified that:

- Creating a post successfully redirected me to the new post's detail page.
- Editing a post updated the content correctly.
- Deleting a post removed it from the list.
- All fetch requests used the correct HTTP methods.

I also tested error handling by submitting empty forms and confirming that error messages were displayed properly. Additionally, I checked the browser Network tab to ensure the frontend was communicating correctly with the backend after fixing the API URL and enabling CORS.

In the end, I tested both normal use cases and edge cases to ensure the application worked correctly

6.3 Load Testing (using loadtest)

PS D:\My_Drive\LAM\Unpx loadtest --concurrency 10 --maxRequests 1000
http://localhost/api/posts?limit=20&offset=0

```
Target URL:          http://localhost/api/posts?limit=20&offset=0
Max requests:        1000
Concurrent clients:  40
Running on cores:    4
Agent:               none

Completed requests:  1000
Total errors:        0
Total time:          3.657 s
Mean latency:        139.9 ms
Effective rps:       273

Percentage of requests served within a certain time
  50%    140 ms
  90%    167 ms
  95%    178 ms
  99%    188 ms
 100%    210 ms (longest request)
```

The server handled 1000 requests with no errors, which shows that it is stable under moderate load. The average latency was about 140 ms, and most requests completed under 180 ms, indicating consistent performance. The effective rate of 273 requests per second suggests the backend and database handled concurrent requests efficiently. Overall, the application remained responsive and did not show performance failures during the test.